

Egglog & Eqsat @ Luminai

Applying eqsat in a production machine learning compiler

About **Luminal**

Series-A startup using eqsat based compiler for ML applications

- Machine learning programs are:
 - very large, expensive tensor algebra programs
 - must be run in a distributed fashion, on many different accelerators
 - are very very commercially valuable
- There is no closed form solution for program optimization, nor accurate cost models
- Lots of performance engineering is achieved via auto-tuning exposed hyper parameters

About our eqsat system

Egglog core embedded in a Rust application

- We have a system which authors egglog at runtime
 - This allows for modular codebase organization, runtime rule selection
- Runs egglog to saturation
- Extracts programs from egglog and profiles them on hardware
- Uses analysis and search algorithms to explore the design space of valid programs

All Thanks to Egglog

Our primary Deggpendency

- egg and egglog being production-ready systems has enabled us to focus on the application layer
- Getting to this point is a massive undertaking, thank you to the **entire** team and community
- An extra special thanks to:
 - Oliver Flatt and Saul Shanabrook who we support developing egglog features.
 - Yihong Zhang and Eli Rosenthal for their volunteer work on egglog performance

EGraph Representation

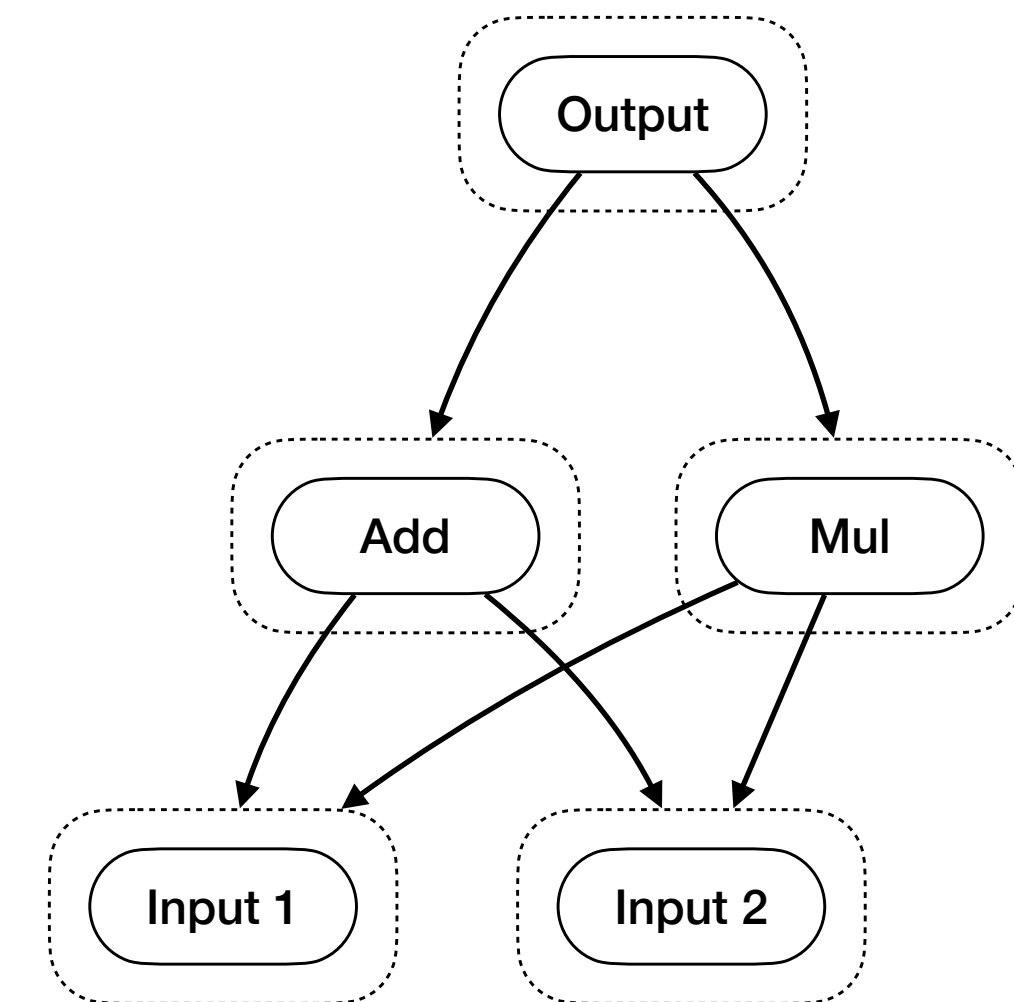
Compact representation of equal terms

- Good at representing sets of equal terms
 - compact when terms are often equal
 - not compact when terms are often unique
- What equality do we care about?
- What should be modeled in the egraph?

Representing Tensor Math

Programs as expressions

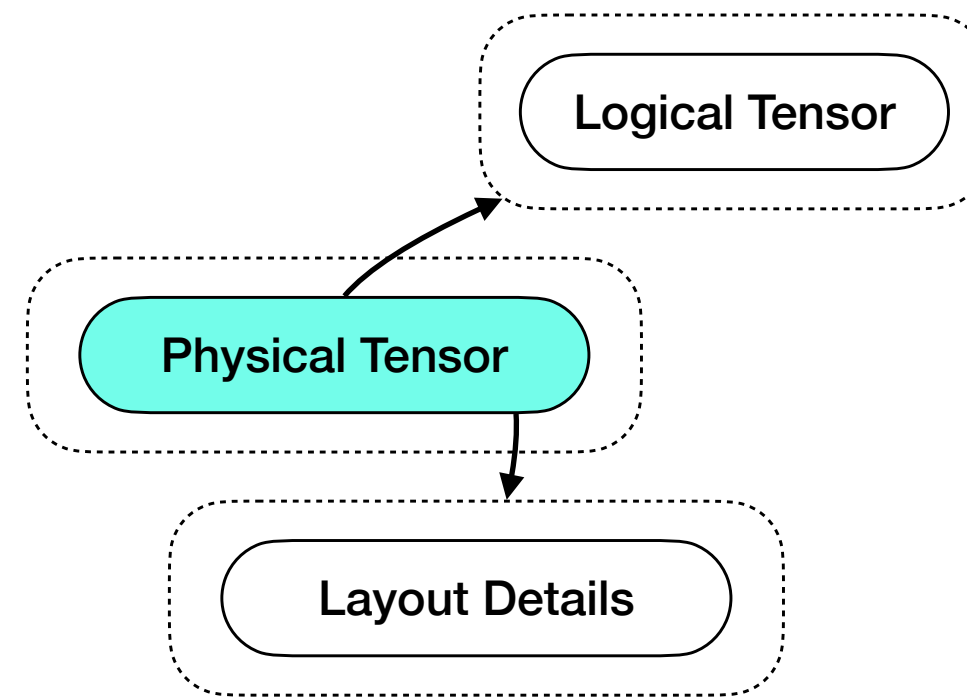
- The outputs of a tensor computation graph can be expressed in terms of the input tensors
- Each logical operation is equivalent to a tensor
- Tensors are uniquely identified by value
- Algebraic tensor value optimizations / rewriting happens at this level (TENSAT)



Representing Physical Tensors

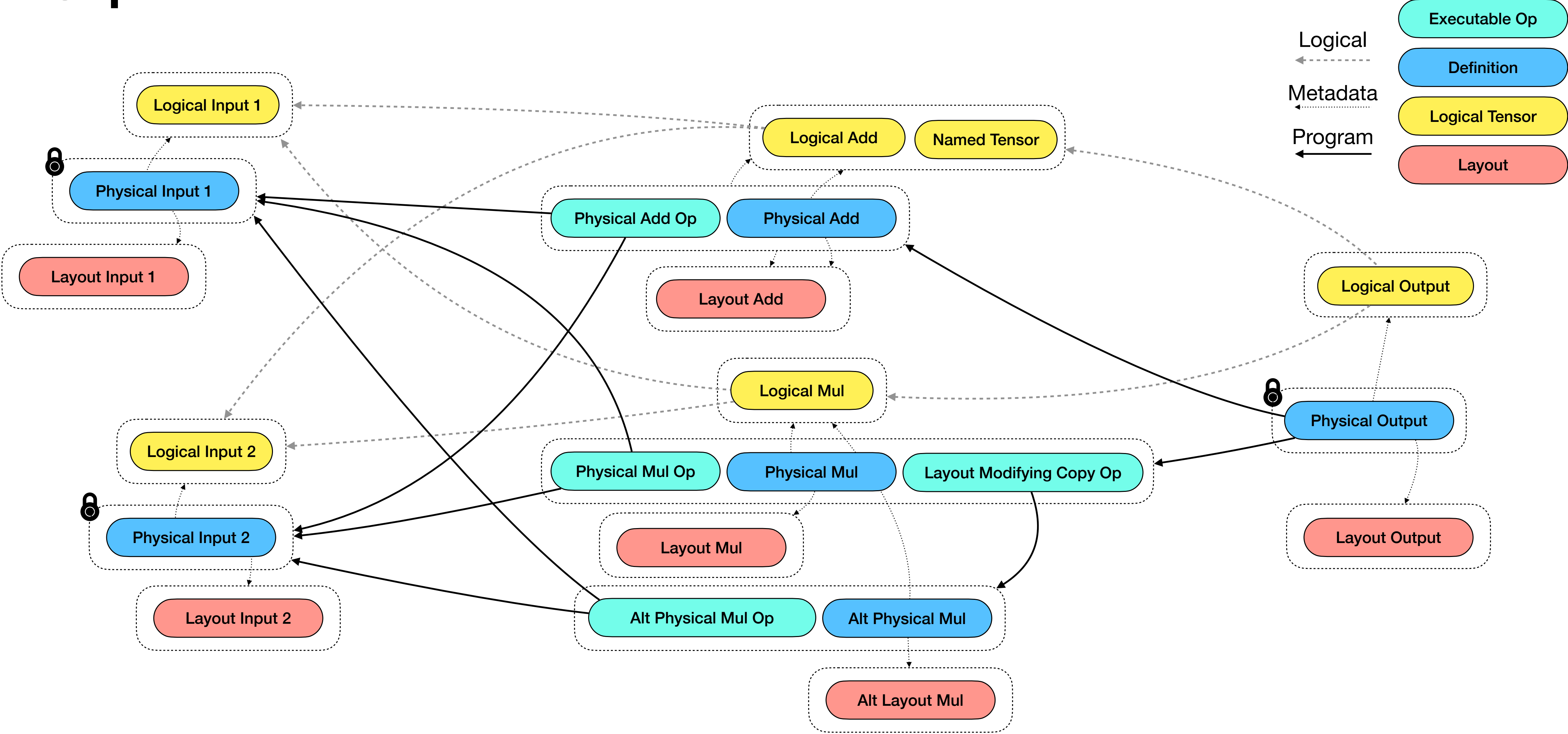
Terms must uniquely identify tensor properties

- Need more than value
 - Memory space
 - Sharding
 - Ownership
 - Layout



Programs Operate on Physical Tensors

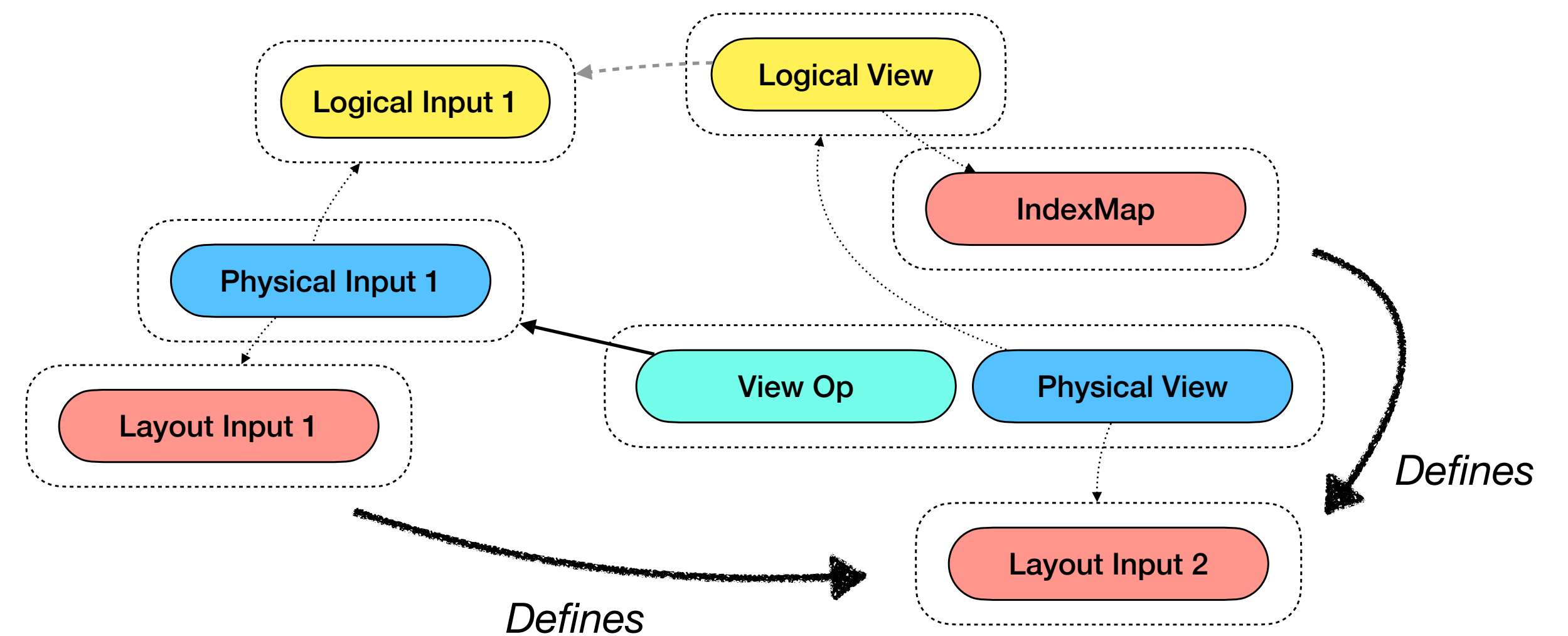
Separation of concerns



Views Operations

Logical Operations implying specific new layouts

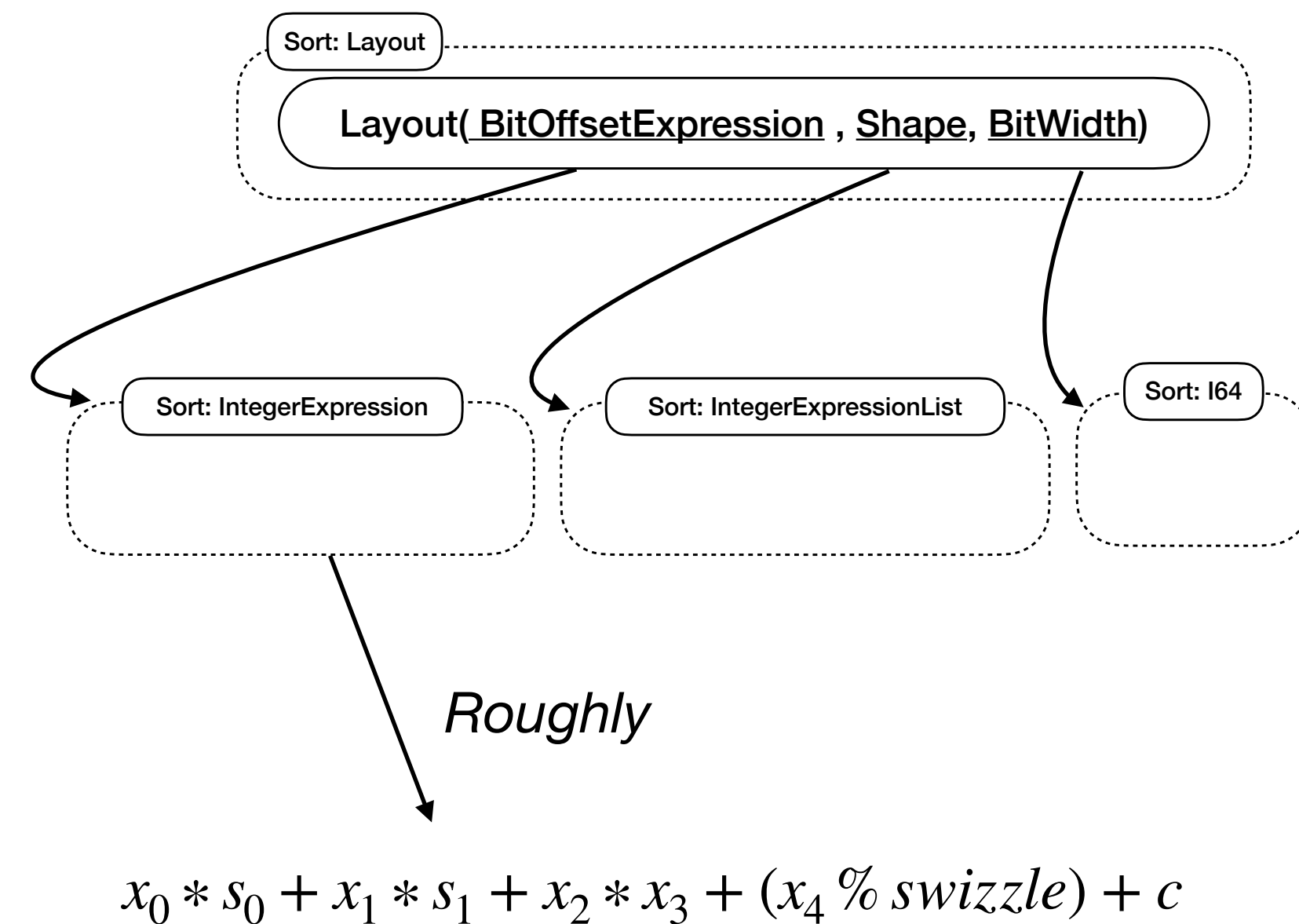
- We have a logical SSA in values
- Logical view operations produce new tensors
 - Broadcast, split dims, slice...
- Physical view operation are required to get them...
- NoOps



Representing Layouts

Layouts as an Expressions

- Our core representation
- memory spaces as bit sequences
 - ML has lots of sub-byte dtypes
- All layouts are total functions from coordinate -> bit-offset
 - Capable of expressing all modern ML formats



Integer Expressions in Egraphs

This is easy

Integer Expressions in Egraphs

This is easy... right?

Integer Expressions in Egraphs

This is easy... right? Right???

Integer Expressions in Egraphs

This is easy... right? Right???



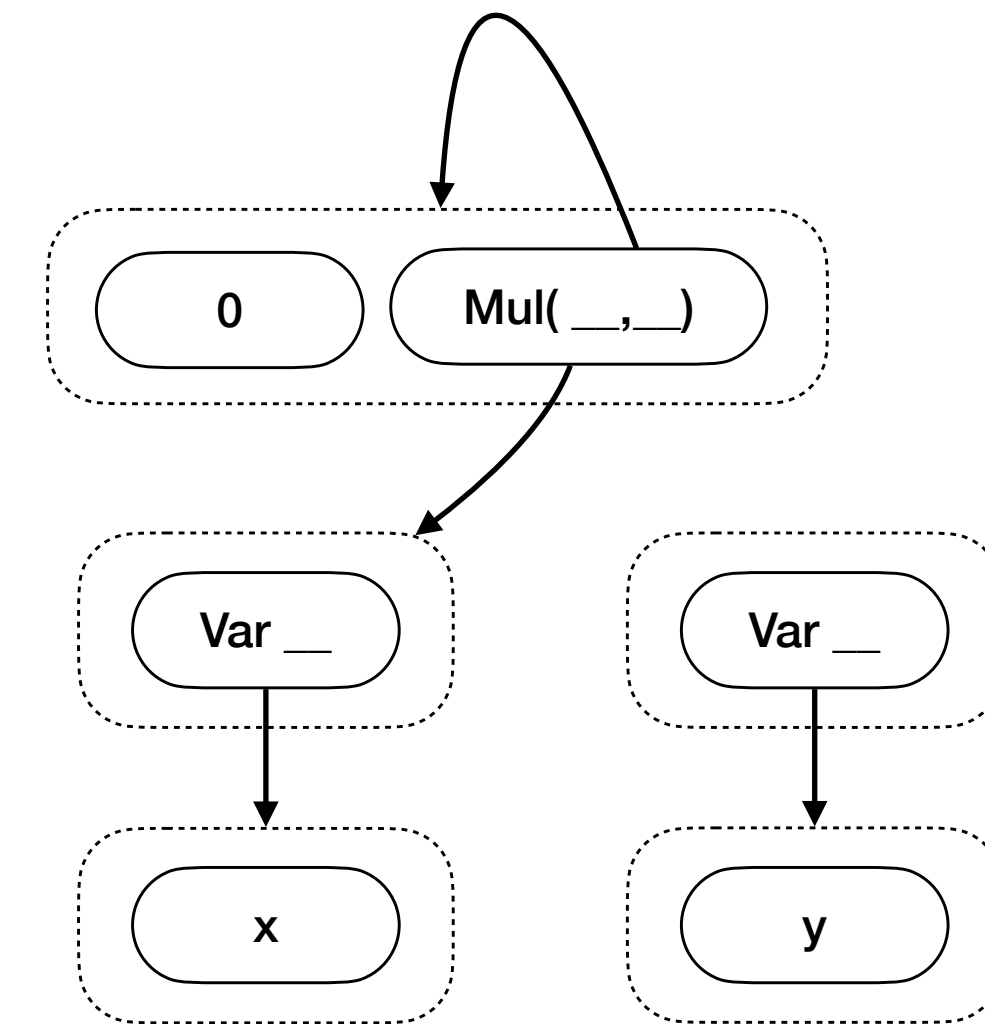
Pattern Matching in Egglog

Matching happens on concrete terms in the egraph

- Egraph can have compact representations of even (some) infinite terms
- Complex patterns can match on these terms... consider:

```
(= ?query (Mul ?term (Mul ?term (0))))
```

- **?var** matches on **x** , not on **y**
- **(Mul (Var ?var) 0) → 0** for all **?var** but you have to actually put **(Mul (Var ?var) 0)** in the egraph to recognize in rule patterns



```
(rule  
  
  ((= ?zero-like (Mul (Var ?var) 0)))  
  
  ((union ?zero-like 0))  
)
```

The Zero Class

Lots of things are zero

- Maybe... we could put a `(Mul (Var ?var) 0)` in the zero class for every `(Var ?var)` so we can recognize these patterns?
- It is very easy to create rulesets which keep putting things into the zero class infinitely.
- This produces no new information and makes everybody sad

Constant Folding... Unfolding?

In this economy?

- We can afford constant folding because it is “normalizing” and terminates
- We could never afford to eagerly unfold values because this does not terminate
- So we can only match the pattern:

`(Add (Num 6) (Num 4))`

with `(Num 10)` iff the class with both

`(Num 10)` and `(Add (Num 6) (Num 4))`
already exists

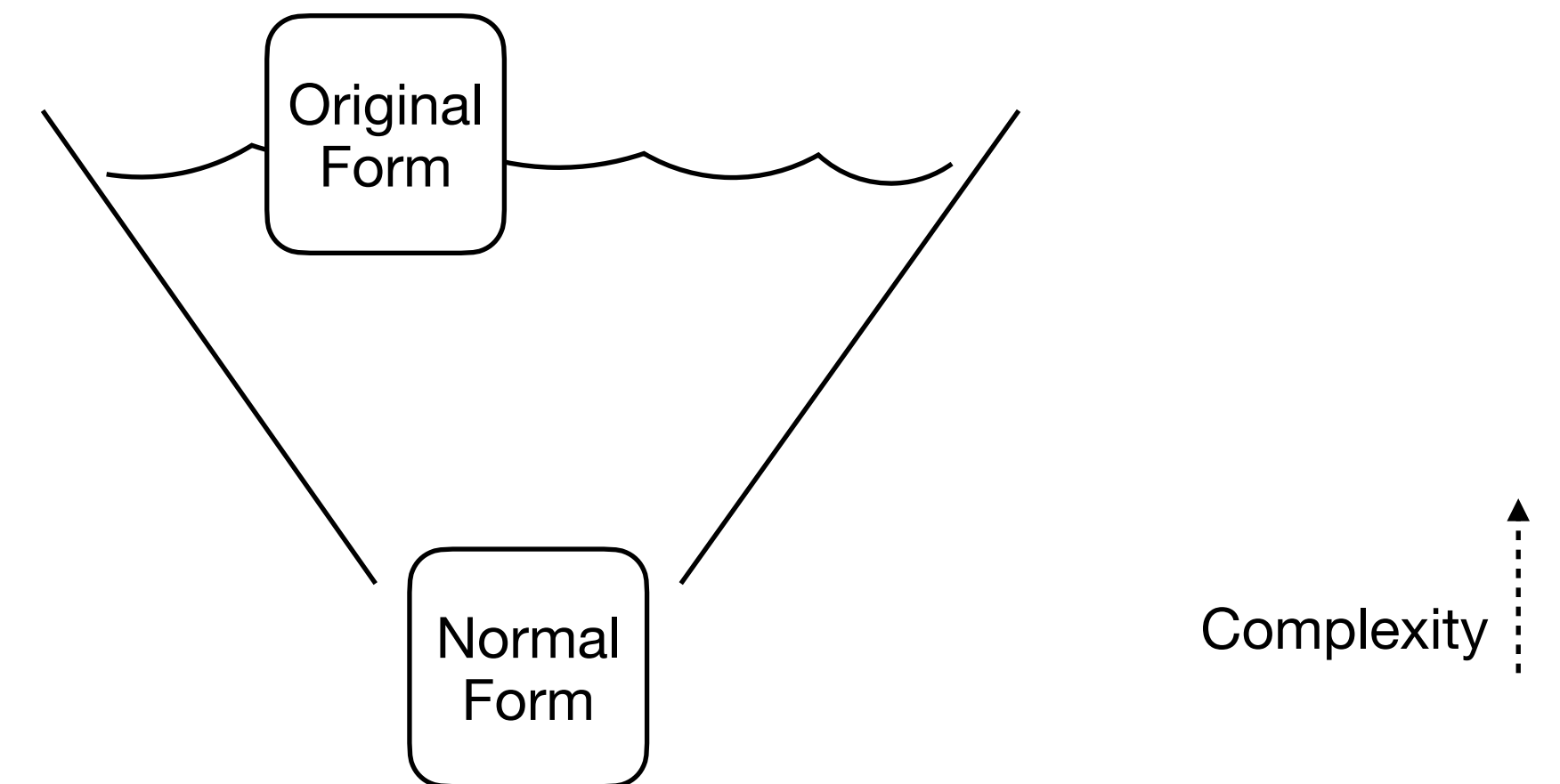
```
(rule
  ((= ?pat (Add (Num ?n1) (Num ?n2))))
  ((union ?pat (?n1 + ?n2)))
)
```

```
(rule
  ((= ?pat (Num ?n3)))
  ((union ?pat (Add (Num ?n3-1) ?1)))
)
```

Don't Complicate Things

Bounded complexity

- We do apply associativity and commutativity
- We non destructively constant fold, and distribute with non-zero guards
- Referencing Phil Zucker's EMT talk
 - We fill in our cone from the initial term, not strictly normalize



“Degenerate” Layouts

0 and 1 are scary numbers

- Many operations require a specific form of layout, like a fixed arity strided form
- If x_0 is in the zero class, s_0 can be anything
- x_0 is in the zero class when the extent of a tensor dimension is 1, this happens all the time
- Recognizing strided forms in degenerate layouts is hard. Patterns only match exactly

Bit Offset Expression

$$(x_0 * s_0) + (x_1 * s_1) + (x_2 * s_2)$$

Strided Layout Form

$$(s_0, s_1, s_2)$$

I almost want a strided form with a hole?

$$(*, s_1, s_2)$$

What isn't in the egraph

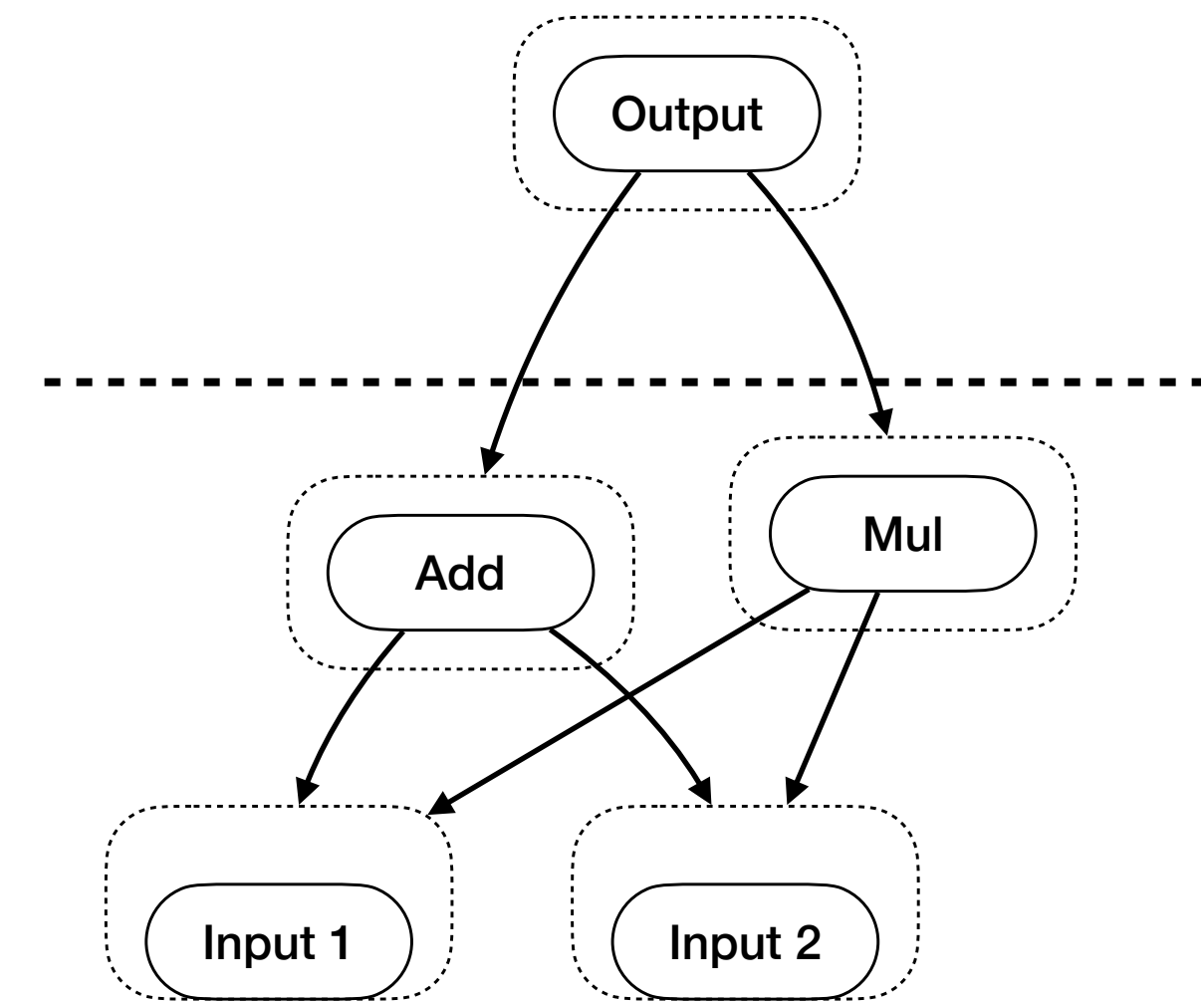
Things that aren't modeled well by expressions

- Bufferization and linearization of resources
- Op Ordering / Explicit Synchronization choices
- Resource usage (memory)
- These are real choices that:
 - need to be costed
 - are mechanical
 - often exhibit strict domination
 - may benefit from not being eagerly enumerated

Lossless Pruning

Operations have lower cost bounds

- Operations must have a theoretical lower cost bound established by:
 - Work performed
 - Memory Moved
- If we find a program with a real cost lower than the theoretic-lower-bound cost for a different program...
 - We can prune without regret



Things we are excited about

What's in the pipeline

- Egraphs / MLIR
- Learning on Egraphs
- Egraph variants
 - AC
 - Slotted
 - Colored/versioned
 - Contextual
 - Egraphs Modulo Theories

We are Hiring

joe@luminal.com